МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«САРАТОВСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИМЕНИ Н. Г. ЧЕРНЫШЕВСКОГО»**

ДВОИЧНОЕ ДЕРЕВО ПОИСКА.

ОТЧЁТ

студентки 3 курса 351 группы
направления 09.03.04 — Программная инженерия
факультета КНиИТ
Винник Маргариты Дмитриевны

Проверено:
доцент, к. ф.-м. н. _____ И. А. Батраева

Саратов 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 Краткая теория 4
1.1 Определения 4
1.2 Представление в памяти 4
1.3 Основные операции 4
1.4 Зависимость времени работы от высоты 4
1.5 Время работы обходов 5
2 Наивная реализация 7
3 Реализация Simple Snippets 17
3.1 Описание реализации 17
3.2 Выявленные недостатки 27
3.3 Способы исправления 29
4 Хорошая, правильная реализация 31
4.1 Устройство класса 37
4.2 Вставка 38
4.3 Поиск 39
4.4 Удаление 39
4.5 Обходы дерева 40
4.6 Обход по уровням и высота 41
4.7 Вывод дерева 41
4.8 Освобождение памяти 41
4.9 Ввод и меню 42
4.10 Оставшиеся ограничения 42
5 Демонстрация работы 43
6 Анализ сложности и расхода памяти 46
6.1 Время работы 46
6.2 Расход памяти 47
6.3 Сравнение реализаций 47
6.4 Вывод 48
ЗАКЛЮЧЕНИЕ 49
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 50

ВВЕДЕНИЕ

Двоичное дерево поиска (англ. *binary search tree*, BST) — структура данных для хранения упорядоченного динамического множества. Она поддерживает поиск, вставку и удаление ключа, а также поиск минимума, максимума, предшественника и преемника [1]. На деревьях поиска, как правило, строятся ассоциативные контейнеры: `std::set` и `std::map` в C++, `TreeSet` в Java.

Время работы основных операций пропорционально высоте дерева h: для сбалансированного дерева $h = \Theta(\lg n)$, а для вырожденного дерева, напоминающего связный список, $h = \Theta(n)$ [1]. Поэтому простое двоичное дерево поиска не даёт гарантий, и на практике используют его самобалансирующиеся версии: красно-чёрные деревья и АВЛ-деревья. Им будут посвящены следующие лабораторные работы.

Цель работы — изучить двоичное дерево поиска, реализовать его на языке C++ и оценить сложность операций и расход памяти.

Задачи:

1. изучить определение двоичного дерева поиска, его свойство и способы обхода;
2. реализовать наивную версию (вставка, поиск, вывод) и проверить её на тестовых задачах;
3. разобрать готовую реализацию Simple Snippets (удаление, три вида обхода, обход по уровням, высота) и выявить её недостатки;
4. написать улучшенную реализацию, устранив выявленные недостатки;
5. оценить сложность операций и расход памяти.

1 Краткая теория

1.1 Определения

Дерево — непустая коллекция вершин и рёбер, в которой любые две вершины соединены ровно одним путём [2]. *Двоичное дерево* — дерево, в котором у каждой вершины не более двух потомков: левый и правый. *Двоичное дерево поиска* — двоичное дерево, в котором для любого узла x ключи левого поддерева меньше $x.key$, а ключи правого поддерева больше [3].

В [1] допускаются равные ключи (в левом поддереве $\leq$, в правом $\geq$), поэтому на рисунке 1.1 есть два узла со значением 5. В нашей реализации все ключи уникальны, поэтому неравенства строгие.

1.2 Представление в памяти

Узел хранит ключ `key` и указатели `left_child` и `right_child`; отсутствующий потомок обозначается `nullptr`. В ряде реализаций в узел добавляют указатель на родителя p [1], но в данной работе информация о родителе не используется. Всё дерево задаётся указателем на корень `root`.

1.3 Основные операции

— **Поиск.** Сравниваем искомый ключ с ключом текущего узла и переходим в левое или правое поддерево, пока не найдём ключ или не дойдём до `nullptr`. Время $O(h)$.

— **Вставка.** Спускаемся так же, как при поиске, и подвешиваем новый узел на месте `nullptr`. Время $O(h)$.

— **Удаление.** Рассматриваются три случая: у узла нет детей (узел просто удаляется); у узла один ребёнок (узел заменяется ребёнком); у узла два ребёнка (на его место ставится преемник — минимальный узел правого поддерева). Время $O(h)$.

— **Обходы.** *Прямой* (узел, левое, правое), *симметричный* (левое, узел, правое; ключи выводятся по возрастанию) и *обратный* (левое, правое, узел). Время $\Theta(n)$ [1].

1.4 Зависимость времени работы от высоты

Одно и то же множество ключей может быть представлено разными деревьями поиска (рисунок 1.1): в варианте (а) высота равна 2, в варианте (б) —

4. Форма дерева определяется порядком вставки ключей: для случайного порядка математическое ожидание высоты равно $O(\lg n)$, а для отсортированной последовательности дерево превращается в цепочку высотой $n - 1$ [1]. Именно поэтому нужны самобалансирующиеся деревья.

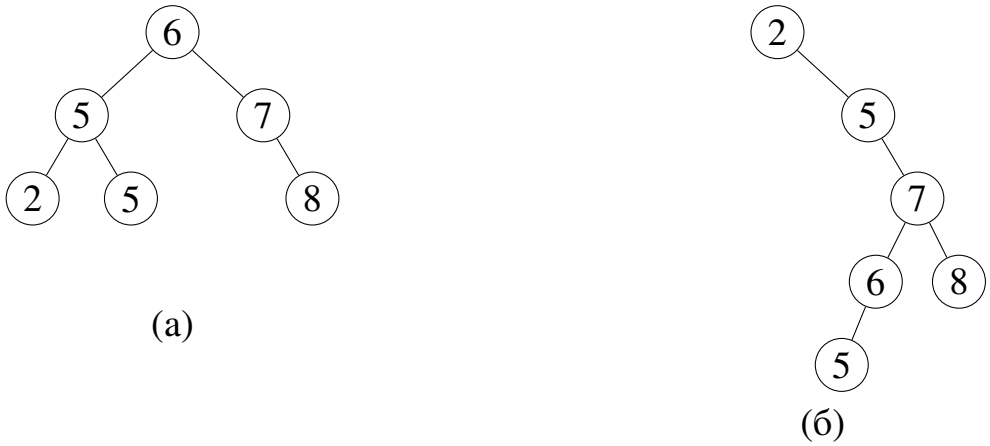


Рисунок 1.1 – Двоичные деревья поиска с одним и тем же набором ключей (по [1])

1.5 Время работы обходов

Все три обхода (прямой, симметричный и обратный) выполняются за время, линейное по числу узлов. Приведём доказательство из [1] для симметричного обхода. Для двух других оно полностью повторяется: процедуры отличаются только тем, где относительно двух рекурсивных вызовов печатается ключ, а на время работы это не влияет.

Процедура симметричного обхода:

```
INORDER-TREE-WALK(x)
1  if x != NIL
2      INORDER-TREE-WALK(x.left)
3      print x.key
4      INORDER-TREE-WALK(x.right)
```

Теорема. Если x — корень поддерева с n узлами, то время работы вызова `INORDER-TREE-WALK`(x) равно $\Theta(n)$.

Доказательство. Пусть $T(n)$ — время вызова для поддерева с n узлами. Процедура посещает все n узлов, поэтому $T(n) = \Omega(n)$. Остаётся показать, что $T(n) = O(n)$.

Для пустого дерева выполняется только проверка $x \neq \mathrm{NIL}$, поэтому $T(0) = c$ для некоторой константы $c > 0$. При $n > 0$ процедура вызывается

для левого поддерева с k узлами и для правого с $n-k-1$ узлами, поэтому

$$T(n) \le T(k) + T(n-k-1) + d,$$

где $d > 0$ — константа, отражающая время выполнения тела процедуры без рекурсивных вызовов.

Методом подстановки докажем, что $T(n) \le (c+d)n + c$. Для $n = 0$ имеем $(c+d) \cdot 0 + c = c = T(0)$. Для $n > 0$:

$$\begin{aligned} T(n) &\le T(k) + T(n-k-1) + d \\ &\le \big((c+d)k + c\big) + \big((c+d)(n-k-1) + c\big) + d \\ &= (c+d)(n-1) + 2c + d \\ &= (c+d)n + c. \end{aligned}$$

Следовательно, $T(n) = O(n)$, и вместе с оценкой снизу получаем $T(n) = \Theta(n)$. $\square$

2 Наивная реализация

Для начала рассмотрим так называемую наивную реализацию двоичного дерева поиска. Определение дерева и его свойства приведены в разделе «Краткая теория». Здесь рассматривается вариант без информации о родителе: узел хранит только ключ `key` и указатели на потомков `left_child` и `right_child`, а само дерево — указатель на корень `root`.

Такая реализация нужна по следующим причинам:

— она даёт представление об устройстве `std::set` в C++, `TreeSet` в Java и других подобных коллекций;

— на ней видно, как на практике работает двоичное дерево поиска (в теории всё выглядит понятным, а детали проявляются только при написании кода);

— она показывает недостатки базовой версии и объясняет, зачем нужна балансировка;

— она служит основой для собственного аналога `std::set`, поддерживающего операции, которых нет в стандартном контейнере.

Реализация является наивной по следующим причинам:

— нет балансировки;

— высота дерева не контролируется: у структуры нет даже поля, хранящего высоту;

— все операции работают за $O(h)$, где h — высота дерева, а в худшем случае $h = O(n)$.

Версия поддерживает вставку, поиск и вывод дерева. Для проверки решены две тестовые задачи; их условия приведены в комментариях к коду.

```
#define _CRT_SECURE_NO_WARNINGS
#include <cstdio>
#include <iostream>
#include <string>

using namespace std;
/*
Правило двоичного дерева поиска:
1) у любой вершины в левом поддереве ключи < ключу в самой вершине
2) у любой вершины в правом поддереве ключи > ключу в самой вершине
```

```
*/

/*class BST {
        struct Node
        {
                int key;
                Node *left_child = nullptr, * right_child = nullptr;
                // конструктор. принимает значение ключа и
                // записывает это значение в поле key
                Node(int k) : key(k) {}
        }
        *root = nullptr;

        bool contains(Node* n, int key) const
        {
                if (!n)
                        return false;
                if (key == n -> key)
                        return true;
                if (key < n->key)
                        return contains(n->left_child, key);
                else
                        return contains(n->right_child, key);
        }
        void insert(Node *n, int key)
        {
                if (!root)
                {
                        root = new Node(key);
                        return;
                }
                if (key == n ->key)
                {
                        return;
                }
```

```
                if (key < n->key)
                {
                        if (n->left_child)
                        {
                                insert(n->left_child, key);
                        }
                        else
                        {
                                n->left_child = new Node(key);
                        }
                }
                else
                {
                        if (n->right_child)
                        {
                                insert(n->right_child, key);
                        }
                        else
                        {
                                n->right_child = new Node(key);
                        }
                }
        }
public:
        bool contains(int key) const
        {
                return contains(root, key);
        }
        void insert(int key)
        {
                insert(root, key);
        }
};
*/
```

```
class BST {
        struct Node
        {
                int key;
                Node* left_child = nullptr, * right_child = nullptr;
                Node(int k) : key(k) {}
        }
        *root = nullptr;

        bool contains(Node* n, int key) const
        {
                if (!n)
                        return false;
                if (key == n->key)
                        return true;
                if (key < n->key)
                        return contains(n->left_child, key);
                else
                        return contains(n->right_child, key);
        }
        // основная функция, от кот. отталкиваемся
        // и модифицируем
        /*void insert(Node*& n, int key)
        {
                if (!n)
                {
                        n = new Node(key);
                        return;
                }
                if (key == n->key)
                {
                        return;
                }
                if (key < n->key)
                {
```

```
                insert(n->left_child, key);
        }
        else
        {
                insert(n->right_child, key);
        }
}
*/
// модификация функции вставки под задачу 2
bool insert(Node*& n, int key)
{
        if (!n)
        {
                n = new Node(key);
                return true;
        }
        if (key == n->key)
                return false;
        if (key < n->key)
                return insert(n->left_child, key);
        else
                return insert(n->right_child, key);
}
// основная функция, от кот.отталкиваемся
// и модифицируем
/*
void print(Node *n) const
{
        if (!n)
                return;
        print(n->left_child);
        if (n->left_child && n->right_child)
        {
                cout << n->key << "\n";
        }
```

```
                //cout << n->key << "\n";
                print(n->right_child);
        }
        */
        void print(Node* n, int level) const
        {
                if (!n)
                        return;
                print(n->left_child, level + 1);
                for (int i = 0; i < level; i++)
                {
                        cout << ".";
                }
                cout << n->key << "\n";
                print(n->right_child, level + 1);
        }
        void destroy(Node *n)
        {
                if (!n)
                        return;
                destroy(n->left_child);
                destroy(n->right_child);
                delete n;
        }
public:
        BST() = default;
        BST(const BST&) = delete;
        BST& operator=(const BST&) = delete;

        bool contains(int key) const
        {
                return contains(root, key);
        }
        bool insert(int key)
        {
```

```
                return insert(root, key);
        }
        /*
        void insert(int key)
        {
                insert(root, key);
        }

        void print() const
        {
                print(root);
        }
        */
        void print() const
        {
                print(root, 0);
        }
        ~BST()
        {
                destroy(root);
        }
};

// для проверки (тестирования кода) решим несколько задач
/*
* -----Задача 1:
* Вводится последовательность целых чисел, оканчивающаяся на нулём.
* Сам ноль в последовательность не входит.
* Постройте в этой последовательности дерева.
* Для полученного дерева выведите список всех вершин,
* имеющих по два ребёнка в порядке возрастания.
* Замечание: если во входных данных есть повторяющие числа, то их нужно
игнорировать, т.е. все элементы дерева уникальны
* Пример:
* 7 3 2 1 9 5 4 6 8 0
```

```
* -----Задача 2:
* Напишите программу, которая будет реализовывать действия в бинарном
* дереве поиска "вставить" и "найти" (по значению).
* Программа должна обрабатывать запросы 3х видов:
* 1) ADD n - если указанного числа ещё нет в дереве, вставлять его и
* выводить слово "DONE", если уже есть - оставлять дерево как было и
* выводить слово "ALREADY".
* 2) SEARCH - следует выводить слово "YES"
* (если значение найдено в дереве) или слово "NO"
* (если не найдено). Дерево при этом не меняется.
* 3) PRINTTREE - выводить всё дерево, обязательно используя алгоритм,
указанный далее в условии задачи.

Алгоритм для вывода дерева:
template<class T> void print_tree(Node<T> *p, int level)
{
        if (p == nullptr)
                return;
        print_tree(p->left, level + 1);
        for(int i = 0; i < level; i++)
                cout << ".";
        cout << p->data << end;
        print_tree(p->right, level + 1);
}
* Изначальный вызов этой функции - print_tree(root, 0)
* Пример:
ADD 2
ADD 3
ADD 2
SEARCH 2
ADD 5
PRINTTREE
SEARCH 7
*/
int main()
```

```
{
        // для задачи 2:
        // Если есть input.txt, то переключаем ввод на него.
        // Раз есть файл с входными данными - значит
  // предполагается, что есть файл и для
  // результата, пишем в него.
        // Если output.txt отсутствует - вывод останется в консоли.
        if (freopen("input.txt", "r", stdin)) {
                (void) freopen("output.txt", "w", stdout);
        }
        // Если input.txt не открылся - freopen вернул nullptr,
        // cin по-прежнему связан с консолью, работаем через неё
        BST bst;
        // для 1й задачи

        /*
        while (true)
        {
                int x;
                cin >> x;
                if (x == 0)
                {
                        break;
                }
                bst.insert(x);
        }
        bst.print();
        */

        string command;
        while (cin >> command)
        {
                if (command == "SEARCH")
                {
                        int key;
```

```
                cin >> key;
                cout << (bst.contains(key) ? "YES\n" :
        "NO\n");
            }
            else if (command == "ADD")
            {
                int key;
                cin >> key;
                cout << (bst.insert(key) ? "DONE\n" :
        "ALREADY\n");
            }
            else if (command == "PRINTTREE")
                bst.print();
            else cout << "Unknown command\n";
        }
}
```

3 Реализация Simple Snippets

3.1 Описание реализации

Следующая рассматриваемая реализация взята из курса Simple Snippets [4]. В отличие от наивной версии, она содержит полный набор операций для работы с деревом: вставку, поиск, удаление, три вида обхода (прямой, симметричный и обратный), обход по уровням, вычисление высоты и вывод дерева, повёрнутого на $90°$. Управление организовано через консольное меню.

Узел представлен классом `TreeNode` с полями `value` (ключ), `left` и `right`. Класс `BST` хранит указатель на корень `root` и содержит следующие методы:

- `insertNode` и `insertRecursive` — две реализации вставки (итеративная и рекурсивная); повторная вставка существующего ключа отклоняется;
- `iterativeSearch` и `recursiveSearch` — две реализации поиска;
- `deleteNode` — рекурсивное удаление, возвращающее указатель на корень поддерева после удаления;
- `minValueNode` — поиск минимального узла в поддереве (нужен при удалении узла с двумя детьми);
- `printPreorder`, `printInorder`, `printPostorder` — три вида обхода;
- `height`, `printGivenLevel`, `printLevelOrderBFS` — высота дерева и обход по уровням;
- `print2D` — вывод дерева "боком".

Код приведён в листинге ниже.

```
#include <iostream>
#define SPACE 10

using namespace std;

class TreeNode {
  public:
    int value;
  TreeNode * left;
  TreeNode * right;

  TreeNode() {
    value = 0;
```

```
    left = NULL;
    right = NULL;
  }
  TreeNode(int v) {
    value = v;
    left = NULL;
    right = NULL;
  }
};

class BST {
  public:
    TreeNode * root;
  BST() {
    root = NULL;
  }
  bool isTreeEmpty() {
    if (root == NULL) {
      return true;
    } else {
      return false;
    }
  }

  void insertNode(TreeNode * new_node) {
    if (root == NULL) {
      root = new_node;
      cout << "Value Inserted as root node!" << endl;
    } else {
      TreeNode * temp = root;
      while (temp != NULL) {
        if (new_node -> value == temp -> value) {
          cout << "Value Already exist," <<
            "Insert another value!" << endl;
          return;
```

```
        } else if ((new_node -> value < temp -> value) &&
        (temp -> left == NULL)) {
          temp -> left = new_node;
          cout << "Value Inserted to the left!" << endl;
          break;
        } else if (new_node -> value < temp -> value) {
          temp = temp -> left;
        } else if ((new_node -> value > temp -> value) &&
        (temp -> right == NULL)) {
          temp -> right = new_node;
          cout << "Value Inserted to the right!" << endl;
          break;
        } else {
          temp = temp -> right;
        }
      }
    }
}
        TreeNode* insertRecursive(TreeNode *r, TreeNode *new_node)
        {
                if(r==NULL)
                {
                        r=new_node;
                        cout <<"Insertion successful"<<endl;
                        return r;
                }

                if(new_node->value < r->value)
                {
                        r->left = insertRecursive(r->left,new_node);
                }
                else if (new_node->value > r->value)
                {
                        r->right = insertRecursive(r->right,new_node);
                }
```

```
        else
        {
                cout << "No duplicate values allowed!" << endl;
                return r;
            }
            return r;
      }

void print2D(TreeNode * r, int space) {
  if (r == NULL) // Base case  1
    return;
  space += SPACE; // Increase distance between levels   2
  print2D(r -> right, space); // Process right child first 3
  cout << endl;
  for (int i = SPACE; i < space; i++) // 5
    cout << " "; // 5.1
  cout << r -> value << "\n"; // 6
  print2D(r -> left, space); // Process left child  7
}

void printPreorder(TreeNode * r) //(current node, Left, Right)
{
  if (r == NULL)
    return;
  /* first print data of node */
  cout << r -> value << " ";
  /* then recur on left sutree */
  printPreorder(r -> left);
  /* now recur on right subtree */
  printPreorder(r -> right);
}

void printInorder(TreeNode * r) //(Left, current node, Right)
{
  if (r == NULL)
```

```
    return;
  /* first recur on left child */
  printInorder(r -> left);
  /* then print the data of node */
  cout << r -> value << " ";
  /* now recur on right child */
  printInorder(r -> right);
}
void printPostorder(TreeNode * r) //(Left, Right, Root)
{
  if (r == NULL)
    return;
  // first recur on left subtree
  printPostorder(r -> left);
  // then recur on right subtree
  printPostorder(r -> right);
  // now deal with the node
  cout << r -> value << " ";
}

TreeNode * iterativeSearch(int v) {
  if (root == NULL) {
    return root;
  } else {
    TreeNode * temp = root;
    while (temp != NULL) {
      if (v == temp -> value) {
        return temp;
      } else if (v < temp -> value) {
        temp = temp -> left;
      } else {
        temp = temp -> right;
      }
    }
    return NULL;
```

```
  }
}

TreeNode * recursiveSearch(TreeNode * r, int val) {
  if (r == NULL || r -> value == val)
    return r;

  else if (val < r -> value)
    return recursiveSearch(r -> left, val);

  else
    return recursiveSearch(r -> right, val);
}

int height(TreeNode * r) {
  if (r == NULL)
    return -1;
  else {
    /* compute the height of each subtree */
    int lheight = height(r -> left);
    int rheight = height(r -> right);

    /* use the larger one */
    if (lheight > rheight)
      return (lheight + 1);
    else return (rheight + 1);
  }
}

/* Print nodes at a given level */
void printGivenLevel(TreeNode * r, int level) {
  if (r == NULL)
    return;
  else if (level == 0)
    cout << r -> value << " ";
```

```
  else // level > 0
  {
    printGivenLevel(r -> left, level - 1);
    printGivenLevel(r -> right, level - 1);
  }
}
void printLevelOrderBFS(TreeNode * r) {
  int h = height(r);
  for (int i = 0; i <= h; i++)
    printGivenLevel(r, i);
}

TreeNode * minValueNode(TreeNode * node) {
  TreeNode * current = node;
  /* loop down to find the leftmost leaf */
  while (current -> left != NULL) {
    current = current -> left;
  }
  return current;
}

TreeNode * deleteNode(TreeNode * r, int v) {
  // base case
  if (r == NULL) {
    return NULL;
  }
  // If the key to be deleted is smaller than the root's key,
  // then it lies in left subtree
  else if (v < r -> value) {
    r -> left = deleteNode(r -> left, v);
  }
  // If the key to be deleted is greater than the root's key,
  // then it lies in right subtree
  else if (v > r -> value) {
    r -> right = deleteNode(r -> right, v);
```

```
    }
    // if key is same as root's key, then
    // This is the node to be deleted
    else {
      // node with only one child or no child
      if (r -> left == NULL) {
        TreeNode * temp = r -> right;
        delete r;
        return temp;
      } else if (r -> right == NULL) {
        TreeNode * temp = r -> left;
        delete r;
        return temp;
      } else {
        // node with two children: Get the inorder successor
        // (smallest in the right subtree)
        TreeNode * temp = minValueNode(r -> right);
        // Copy the inorder successor's content to this node
        r -> value = temp -> value;
        // Delete the inorder successor
        r -> right = deleteNode(r -> right, temp -> value);
        //deleteNode(r->right, temp->value);
      }
    }
    return r;
  }

};

int main() {
  BST obj;
  int option, val;

  do {
    cout << "What operation do you want to perform? " <<
```

```
  " Select Option number. Enter 0 to exit." << endl;
cout << "1. Insert Node" << endl;
cout << "2. Search Node" << endl;
cout << "3. Delete Node" << endl;
cout << "4. Print/Traversal BST values" << endl;
cout << "5. Height of Tree" << endl;
cout << "6. Clear Screen" << endl;
cout << "0. Exit Program" << endl;

cin >> option;
//Node n1;
TreeNode * new_node = new TreeNode();

switch (option) {
case 0:
  break;
case 1:
          cout <<"INSERT"<<endl;
          cout <<"Enter VALUE of TREE NODE to INSERT in BST: ";
          cin >> val;
          new_node->value = val;
          obj.root= obj.insertRecursive(obj.root,new_node);
          //obj.insertNode(new_node);
          cout<<endl;
                break;

case 2:
  cout << "SEARCH" << endl;
  cout << "Enter VALUE of TREE NODE to SEARCH in BST: ";
  cin >> val;
  //new_node = obj.iterativeSearch(val);
  new_node = obj.recursiveSearch(obj.root, val);
  if (new_node != NULL) {
    cout << "Value found" << endl;
  } else {
```

```
      cout << "Value NOT found" << endl;
    }
    break;
  case 3:
    cout << "DELETE" << endl;
    cout << "Enter VALUE of TREE NODE to DELETE in BST: ";
    cin >> val;
    new_node = obj.iterativeSearch(val);
    if (new_node != NULL) {
      obj.deleteNode(obj.root, val);
      cout << "Value Deleted" << endl;
    } else {
      cout << "Value NOT found" << endl;
    }
    break;
  case 4:
    cout << "PRINT 2D: " << endl;
    obj.print2D(obj.root, 5);
    cout << endl;
    cout << "Print Level Order BFS: \n";
    obj.printLevelOrderBFS(obj.root);
    cout << endl;
    //              cout <<"PRE-ORDER: ";
    //              obj.printPreorder(obj.root);
    //              cout<<endl;
    //              cout <<"IN-ORDER: ";
    //              obj.printInorder(obj.root);
    //              cout<<endl;
    //              cout <<"POST-ORDER: ";
    //              obj.printPostorder(obj.root);
    break;
  case 5:
    cout << "TREE HEIGHT" << endl;
    cout << "Height : " << obj.height(obj.root) << endl;
    break;
```

```
    case 6:
      system("cls");
      break;
    default:
      cout << "Enter Proper Option number " << endl;
    }

  } while (option != 0);

  return 0;
}
```

3.2 Выявленные недостатки

Реализация работает на типичных сценариях, но содержит ошибки и проектные недостатки. Часть из них проверена запуском программы под AddressSanitizer и LeakSanitizer (компилятор g++).

1. **Потеря нового корня при удалении (ошибка).** В пункте меню «Удалить» результат вызова `obj.deleteNode(obj.root, val)` отбрасывается, хотя функция возвращает указатель на корень поддерева после удаления. Если удаляется корень, у которого нет одного из потомков, память освобождается оператором `delete`, а `obj.root` продолжает указывать на неё, и любая следующая операция обращается к освобождённой памяти. Проверка: вставка ключа и его удаление из дерева из одного узла с последующим выводом дают ошибку `heap-use-after-free`.
2. **Утечки памяти.**
   (а) В начале каждой итерации цикла меню безусловно выполняется `new TreeNode()`, хотя узел нужен только при вставке.
   (б) В пунктах «Найти» и «Удалить» указатель `new_node` перезаписывается результатом поиска, и созданный узел теряется.
   (в) При вставке дубликата новый узел не освобождается.
   (г) У класса `BST` нет деструктора, поэтому при выходе из программы дерево не освобождается. Проверка: после вставки трёх ключей и немедленного выхода LeakSanitizer сообщает об утечке 96 байт в четырёх блоках — три узла дерева и узел, созданный на последней итерации меню.
3. **Отсутствие проверки ввода.** Значения читаются без проверки резуль-

тата. При вводе нечислового значения поток `cin` переходит в состояние ошибки, последующие операции чтения не изменяют переменную `option`, и программа бесконечно повторяет последний выбранный пункт. Проверка: при вводе букв вместо ключа ключ 0 вставляется один раз, после чего программа бесконечно выводит сообщение о дубликате.

4. **Дублирование кода и двойной поиск.** Есть две реализации вставки и две — поиска, при этом в меню используется только по одной (остальные закомментированы или не вызываются).

   При удалении сначала вызывается `iterativeSearch`, чтобы проверить наличие ключа, а затем `deleteNode` ищет тот же ключ повторно.

5. **Смешение логики и ввода-вывода.** Методы вставки сами печатают сообщения, поэтому результат операции нельзя использовать в программе (например, в тесте) и нельзя повторно использовать класс в другой программе. Кроме того, узел создаёт вызывающий код, и неясно, кто отвечает за его освобождение.

6. **Нарушение инкапсуляции.** Все поля и вспомогательные рекурсивные методы объявлены `public`. Чтобы вставить ключ, приходится писать `obj.insertRecursive(obj.root, new_node)`, и пользователь класса может случайно нарушить свойство дерева, изменив `root` или указатели узлов.

7. **Неэффективный обход по уровням.** `printLevelOrderBFS` для каждого уровня заново вызывает `printGivenLevel`, который обходит дерево с корня. Суммарное время составляет $O(n \cdot h)$, а для вырожденного дерева $O(n^2)$. Название BFS вводит в заблуждение: очереди нет, по сути выполняется повторяющийся поиск в глубину.

8. **Устаревший и непереносимый стиль.** Используются `NULL` вместо `nullptr`, макрос `#define SPACE`, очистка экрана через `system("cls")` (работает только в Windows); методы, не изменяющие дерево, не помечены `const`; копирование объекта не запрещено.

9. **Глубина рекурсии.** Рекурсивная вставка, удаление, обходы и вычисление высоты используют стек, глубина которого равна высоте дерева. Для вырожденного дерева из очень большого числа узлов это грозит переполнением стека. Недостаток свойственен всем рекурсивным реализациям и обсуждается в разделе 6.

3.3 Способы исправления

Для каждого недостатка ниже указан способ его устранения; реализация с этими исправлениями описана в разделе 4.

Таблица 3.1 – Недостатки реализации Simple Snippets и их исправление

№	Недостаток	Исправление
1	Потеря нового корня при удалении	Передавать указатель по ссылке (`Node*&`): присваивание `node = ...` изменяет указатель в родителе или сам `root`, возвращать значение не нужно.
2	Утечки памяти	Узлы создаёт и удаляет только сам класс: `insert(int key)` создаёт узел внутри, лишних `new` в меню нет; деструктор освобождает дерево обратным обходом; копирование запрещено (`= delete`).
3	Нет проверки ввода	Функция `readInt` проверяет результат `cin >> value`; при ошибке сбрасывает флаги (`cin.clear()`), пропускает остаток строки (`cin.ignore`) и выводит сообщение.
4	Дублирование кода, двойной поиск	Оставлен один вариант каждой операции; `remove` находит и удаляет узел за один спуск и возвращает `bool`.
5	Логика смешана с выводом	Методы возвращают `bool` (выполнено или нет), печатает только `main`; в метод вставки передаётся ключ, а не узел.
6	Нарушена инкапсуляция	`root`, `Node` и рекурсивные помощники `private`; публичный интерфейс состоит из десяти методов.
7	Обход по уровням за $O(n \cdot h)$	Использовать очередь `std::queue`: каждый узел помещается в неё и извлекается ровно один раз, время $O(n)$.
8	Устаревший стиль	`nullptr` вместо `NULL`; макрос заменён выражением; `system("cls")` удалён; методы, не меняющие дерево, помечены `const`.

№	Недостаток	Исправление
9	Глубина рекурсии	Не устранён: поиск сделан итеративным, остальные операции остаются рекурсивными (см. подраздел 4.10).

4 Хорошая, правильная реализация

Улучшенная реализация сохраняет возможности версии Simple Snippets (вставка, поиск, удаление, три вида обхода, обход по уровням, высота, вывод дерева, меню), но устраняет недостатки, перечисленные в таблице 3.1. Весь код приведён в листинге ниже, далее разобрано, как он работает.

```
// Двоичное дерево поиска (BST) на C++17.
// Без windows.h: код работает на любой ОС.

#include <iostream>
#include <limits>
#include <queue>
#include <string>
using namespace std;

class BST {
public:
    BST() = default;
    ~BST() { clear(root); } // освобождаем всё дерево

    // Копировать дерево запрещаем: иначе два объекта
        // владели бы одними узлами
    BST(const BST&) = delete;
    BST& operator=(const BST&) = delete;

    // ---- Основные операции. Возвращают true/false,
        // ничего не печатают ----
    bool insert(int key) { return insertHelper(root, key); }
    bool remove(int key) { return removeHelper(root, key); }
    bool contains(int key) const
        {
                return findHelper(root, key) != nullptr;
        }
    bool empty() const { return root == nullptr; }
```

```
// ---- Три обхода ----
void preorder() const
    {
            preorderHelper(root);
            cout << '\n';
    }   // узел, левое, правое
void inorder() const
    {
            inorderHelper(root);
            cout << '\n';
    }   // левое, узел, правое
void postorder() const
    {
            postorderHelper(root);
            cout << '\n';
    }   // левое, правое, узел

// ---- Обход по уровням (через очередь, за O(n)) ----
void levelOrder() const {
    if (!root) { cout << '\n'; return; }
    queue<Node*> q;
    q.push(root);
    while (!q.empty()) {
        Node* cur = q.front();
        q.pop();
        cout << cur->key << ' ';
        if (cur->left)  q.push(cur->left);
        if (cur->right) q.push(cur->right);
    }
    cout << '\n';
}
// Высота в рёбрах: пустое дерево -1, одиночный узел 0
int height() const { return heightHelper(root); }
// Дерево, повёрнутое на 90 градусов: корень слева,
    // правые дети сверху
```

```
    void print2D() const {
        if (!root) { cout << "(пусто)\n"; return; }
        print2DHelper(root, 0);
    }

private:
    struct Node {
        int key;
        Node* left = nullptr;
        Node* right = nullptr;
        explicit Node(int k) : key(k) {}
    };

    Node* root = nullptr;

    // ---- Вставка ----
    // Node*& значит "ссылка на сам указатель".
    // Присвоив node = new Node(...),
    // мы меняем тот указатель, который лежит в родителе (или root).
    bool insertHelper(Node*& node, int key) {
        if (!node) {        // нашли пустое место
            node = new Node(key);
            return true;
        }
        if (key < node->key)
                        return insertHelper(node->left, key);
        if (key > node->key)
                        return insertHelper(node->right, key);
        return false; // такой ключ уже есть
    }

    // ---- Поиск ----
    Node* findHelper(Node* node, int key) const {
        while (node && node->key != key)
            node = (key < node->key) ? node->left : node->right;
```

```
    return node;        // nullptr, если не нашли
}

// ---- Удаление ----
bool removeHelper(Node*& node, int key) {
    if (!node) return false;        // не нашли
    if (key < node->key) return removeHelper(node->left, key);
    if (key > node->key) return removeHelper(node->right, key);

    // Нашли удаляемый узел.
            // нет левого: заменяем правым (или nullptr)
    if (!node->left) {
        Node* old = node;
        node = node->right;
        delete old;
    }
            // нет правого: заменяем левым
    else if (!node->right) {
        Node* old = node;
        node = node->left;
        delete old;
    }
    else {// два ребёнка
                    // преемник = минимум правого поддерева
        Node* succ = node->right;
        while (succ->left) succ = succ->left;
        node->key = succ->key;        // копируем его ключ сюда
                    // и удаляем преемника (у него нет левого)
        removeHelper(node->right, succ->key);
    }
    return true;
}

// ---- Обходы ----
void preorderHelper(const Node* node) const {
```

```
    if (!node) return;
    cout << node->key << ' ';
    preorderHelper(node->left);
    preorderHelper(node->right);
}
void inorderHelper(const Node* node) const {
    if (!node) return;
    inorderHelper(node->left);
    cout << node->key << ' ';
    inorderHelper(node->right);
}
void postorderHelper(const Node* node) const {
    if (!node) return;
    postorderHelper(node->left);
    postorderHelper(node->right);
    cout << node->key << ' ';
}

int heightHelper(const Node* node) const {
    if (!node) return -1;
    int l = heightHelper(node->left);
    int r = heightHelper(node->right);
    return 1 + (l > r ? l : r);
}

void print2DHelper(const Node* node, int depth) const {
    if (!node) return;
            // сначала правое: оно окажется сверху
    print2DHelper(node->right, depth + 1);
    cout << string(depth * 4, ' ') << node->key << '\n';
    print2DHelper(node->left, depth + 1);
}

// Удаление всего поддерева обратным обходом:
    // сначала дети, потом сам узел
```

```
    void clear(Node* node) {
        if (!node) return;
        clear(node->left);
        clear(node->right);
        delete node;
    }
};

// Читает целое число; при неверном вводе очищает поток
// и возвращает false
bool readInt(const char* prompt, int& value) {
    cout << prompt;
    if (cin >> value) return true;
    if (cin.eof()) exit(0);        // конец ввода: выходим
    cin.clear();
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
    cout << "Нужно ввести целое число.\n";
    return false;
}

int main()
{
    setlocale(LC_ALL, "Russian");
    BST tree;
    int option = -1;

    while (option != 0) {
        cout << "\n1. Вставить  2. Найти  3. Удалить
                4. Обходы и вывод  5. Высота  0. Выход\n";
        if (!readInt("> ", option)) { option = -1; continue; }

        int val;
        switch (option) {
        case 0:
            break;
```

```
            case 1:
                if (readInt("Ключ: ", val))
                    cout << (tree.insert(val) ? "Вставлено\n" :
                                                "Такой ключ уже есть\n");
                break;
            case 2:
                if (readInt("Ключ: ", val))
                    cout << (tree.contains(val) ? "Найден\n" :
                                                  "Не найден\n");
                break;
            case 3:
                if (readInt("Ключ: ", val))
                    cout << (tree.remove(val) ? "Удалён\n" : "Не найден\n");
                break;
            case 4:
                cout << "Прямой:       "; tree.preorder();
                cout << "Симметричный: "; tree.inorder();
                cout << "Обратный:     "; tree.postorder();
                cout << "По уровням:   "; tree.levelOrder();
                cout << "Дерево:\n";     tree.print2D();
                break;
            case 5:
                cout << "Высота: " << tree.height() << '\n';
                break;
            default:
                cout << "Нет такого пункта\n";
            }
        }
        return 0;
}
```

4.1 Устройство класса

Класс BST делится на открытый интерфейс и закрытую часть. Снаружи видны только операции над деревом, а узлы, корень и рекурсивные помощники скрыты.

Таблица 4.1 – Открытый интерфейс класса `BST`

Метод	Назначение	Результат
`insert(key)`	вставка ключа; повторный ключ отклоняется	`bool`
`remove(key)`	удаление узла с заданным ключом	`bool`
`contains(key)`	проверка наличия ключа	`bool`
`empty()`	проверка, что дерево пусто	`bool`
`preorder()`, `inorder()`, `postorder()`	прямой, симметричный и обратный обходы с печатью ключей	`void`
`levelOrder()`	обход по уровням (в ширину)	`void`
`height()`	высота дерева в рёбрах (пустое дерево -1)	`int`
`print2D()`	вывод дерева, повёрнутого на 90°	`void`

Узел `Node` описан внутри класса как закрытая структура с ключом и двумя указателями на потомков. Корень `root` также закрыт, поэтому все узлы создаются и удаляются только методами самого класса. Это исключает вопрос о владении памятью, который в версии Simple Snippets оставался нерешённым.

4.2 Вставка

Вставка выполнена рекурсивной функцией, принимающей указатель на узел *по ссылке*:

```
bool insertHelper(Node*& node, int key) {
    if (!node) {                                // нашли пустое место
        node = new Node(key);
        return true;
    }
    if (key < node->key) return insertHelper(node->left, key);
    if (key > node->key) return insertHelper(node->right, key);
    return false;                               // такой ключ уже есть
}
```

Запись `Node*&` означает ссылку на сам указатель, а не на узел, на который он указывает. Поэтому присваивание `node = new Node(key)` изменяет то поле,

которое было передано: `root` либо `left`/`right` родителя. Отдельно хранить указатель на родителя и возвращать новый корень поддерева не требуется.

Пример: вставка ключа 10 в дерево с ключами 7, 3, 2, 1, 9, 5, 4, 6, 8. Так как $10 > 7$, вызов переходит к правому потомку; затем $10 > 9$, и вызов получает ссылку на правое поле узла 9, которое равно `nullptr`. Там создаётся узел 10, и он оказывается правым потомком девятки. Вставка уже имеющегося ключа доходит до последней строки и возвращает `false`, дерево не меняется.

4.3 Поиск

Поиск выполнен итеративно и не использует стек вызовов:

```
Node* findHelper(Node* node, int key) const {
    while (node && node->key != key)
        node = (key < node->key) ? node->left : node->right;
    return node;                                // nullptr, если не нашли
}
```

Цикл идёт вниз, пока не встретит искомый ключ или пустой указатель. Метод `contains` возвращает `true`, если результат не равен `nullptr`.

4.4 Удаление

Удаление использует ту же идею ссылки на указатель и рассматривает три случая [1]:

```
bool removeHelper(Node*& node, int key) {
    if (!node) return false;                    // не нашли
    if (key < node->key) return removeHelper(node->left, key);
    if (key > node->key) return removeHelper(node->right, key);

    if (!node->left) {                          // нет левого ребёнка
        Node* old = node;
        node = node->right;
        delete old;
    } else if (!node->right) {                  // нет правого ребёнка
        Node* old = node;
        node = node->left;
        delete old;
    } else {                                    // два ребёнка
        Node* succ = node->right;               // преемник: минимум справа
        while (succ->left) succ = succ->left;
```

```
        node->key = succ->key;                  // копируем его ключ
        removeHelper(node->right, succ->key); // удаляем преемника
    }
    return true;
}
```

— **Нет левого ребёнка** (в том числе лист). Узел заменяется своим правым потомком, а если его нет, то `nullptr`. Так как `node` — ссылка на указатель родителя, замена выполняется одним присваиванием.

— **Нет правого ребёнка.** Симметрично, узел заменяется левым потомком. Пример: узел 9 в дереве выше имеет единственного потомка 8, который встаёт на его место.

— **Два ребёнка.** Нельзя просто убрать узел, поэтому на его место ставится преемник — следующий по величине ключ, то есть минимум правого поддерева. Он достигается шагом вправо и спуском влево до конца. Ключ преемника копируется в удаляемый узел, после чего сам преемник удаляется из правого поддерева; у него нет левого ребёнка, поэтому это один из двух предыдущих случаев.

Пример. При удалении ключа 5 (дети 4 и 6) преемником становится 6: его ключ записывается в узел, который раньше хранил 5, а старый узел 6 удаляется. При удалении корня 7 преемником становится 8 — минимум правого поддерева, корнем которого служит 9. Так как удаление корня тоже выполняется через ссылку на `root`, дерево остаётся корректным, в отличие от версии Simple Snippets.

4.5 Обходы дерева

Три вида обхода отличаются только положением печати ключа относительно двух рекурсивных вызовов:

— прямой (`preorder`): узел, левое поддерево, правое поддерево;

— симметричный (`inorder`): левое поддерево, узел, правое поддерево; ключи выводятся по возрастанию, что используется как проверка корректности дерева;

— обратный (`postorder`): левое поддерево, правое поддерево, узел.

Для дерева с ключами 7, 3, 2, 1, 9, 5, 4, 6, 8 они выводят `7 3 2 1 5 4 6 9 8`, `1 2 3 4 5 6 7 8 9` и `1 2 4 6 5 3 8 9 7` соответственно. Время каждого

обхода равно $\Theta(n)$, как показано в теореме о времени центрированного обхода из раздела «Краткая теория».

4.6 Обход по уровням и высота

Обход по уровням выполнен с помощью очереди:

```
queue<Node*> q;
q.push(root);
while (!q.empty()) {
    Node* cur = q.front();
    q.pop();
    cout << cur->key << ' ';
    if (cur->left)  q.push(cur->left);
    if (cur->right) q.push(cur->right);
}
```

Сначала в очередь помещается корень. На каждом шаге из очереди извлекается узел, печатается его ключ, а его потомки добавляются в конец. Так узлы выводятся уровень за уровнем, слева направо. Каждый узел помещается в очередь и извлекается ровно один раз, поэтому время равно $O(n)$, в отличие от $O(n \cdot h)$ в версии Simple Snippets. Для того же дерева результат: `7 3 9 2 5 8 1 4 6`.

Высота считается рекурсивно как единица плюс большая из высот поддеревьев и измеряется в рёбрах: пустое дерево имеет высоту -1, дерево из одного узла — 0. Такое соглашение совпадает с принятым в [1].

4.7 Вывод дерева

Метод `print2D` выводит дерево, повёрнутое на $90°$: корень находится слева, правые потомки — выше, левые — ниже. Для этого выполняется обход «правое поддерево, узел, левое поддерево», а перед каждым ключом печатается отступ в $4 \cdot d$ пробелов, где d — глубина узла. Благодаря этому форма дерева видна сразу, без `windows.h`, и программа работает на любой операционной системе.

4.8 Освобождение памяти

Деструктор вызывает функцию `clear`, которая обходит дерево в обратном порядке: сначала удаляются оба поддерева, затем сам узел. Порядок важен, поскольку после `delete` указатели на потомков уже недоступны. Копирование

объекта запрещено (`BST(const BST&) = delete` и такой же оператор присваивания), иначе два объекта владели бы одними и теми же узлами и попытались бы удалить их дважды.

4.9 Ввод и меню

Функция `readInt` читает целое число и проверяет результат: при неверном вводе она сбрасывает состояние потока, пропускает остаток строки, сообщает об ошибке и возвращает `false`, а при конце ввода завершает программу. В отличие от версии Simple Snippets, меню продолжает работать после неверного ввода. Операции вызываются через `switch`; сообщения печатает только `main`, а методы класса возвращают `bool`.

4.10 Оставшиеся ограничения

— Вставка, удаление, обходы, высота и вывод остаются рекурсивными, поэтому глубина стека равна высоте дерева. Поиск итеративный. Рекурсию можно заменить циклом или явным стеком, но это усложнило бы код, и в данной работе не делается.
— Дерево не балансируется: форма зависит от порядка вставки (раздел 6).
— Ключи имеют тип `int`; для произвольного типа класс можно сделать шаблоном.
— Высота не хранится в узлах и вычисляется заново при каждом вызове за $\Theta(n)$.

5 Демонстрация работы

Ниже приведены результаты работы улучшенной реализации. Пункты меню в листингах опущены, показан только результат выбранных операций.

1. Построение дерева. В пустое дерево вставляются ключи 7, 3, 2, 1, 9, 5, 4, 6, 8, затем выбирается пункт «Обходы и вывод»:

```
Прямой:       7 3 2 1 5 4 6 9 8
Симметричный: 1 2 3 4 5 6 7 8 9
Обратный:     1 2 4 6 5 3 8 9 7
По уровням:   7 3 9 2 5 8 1 4 6
Дерево:
    9
        8
7
            6
        5
            4
    3
        2
            1
```

Симметричный обход выдаёт ключи по возрастанию, значит, свойство дерева поиска выполнено. Высота дерева, как показывает пункт «Высота», равна 3.

2. Повторная вставка и поиск. Попытка вставить ключ 5 повторно отклоняется, поиск находит ключ 6 и не находит ключ 10:

```
Вставка 5:  Такой ключ уже есть
Поиск 6:    Найден
Поиск 10:   Не найден
```

3. Удаление узла с двумя детьми (ключ 5). Преемником становится ключ 6, он занимает место удалённого узла:

```
Удалён
Прямой:       7 3 2 1 6 4 9 8
Симметричный: 1 2 3 4 6 7 8 9
Обратный:     1 2 4 6 3 8 9 7
По уровням:   7 3 9 2 6 8 1 4
Дерево:
```

```
    9
        8
7
        6
            4
    3
        2
            1
```

4. Удаление корня (ключ 7). Новым корнем становится преемник 8:

```
Удалён
Прямой:       8 3 2 1 6 4 9
Симметричный: 1 2 3 4 6 8 9
Обратный:     1 2 4 6 3 9 8
По уровням:   8 3 9 2 6 1 4
Дерево:
    9
8
        6
            4
    3
        2
            1
```

5. Удаление листа и несуществующего ключа. Ключ 1 (лист) удаляется, ключ 100 не найден, после чего высота остаётся равной 3, а симметричный обход по-прежнему упорядочен:

```
Удаление 1:    Удалён
Удаление 100:  Не найден
Высота:        3
Симметричный:  2 3 4 6 8 9
```

6. Неверный ввод. Если вместо ключа ввести буквы, программа сообщает об ошибке и продолжает работу (в версии Simple Snippets она зацикливалась):

```
Ключ: abc
Нужно ввести целое число.
```

7. Вырождение дерева. Если вставлять ключи 1, 2, 3, 4, 5 в порядке возрастания, дерево превращается в цепочку, и его высота равна 4 (на единицу меньше числа узлов):

```
          5
        4
      3
    2
1
```

Последний пример объясняет, зачем нужны самобалансирующиеся деревья, которым посвящены следующие работы.

6 Анализ сложности и расхода памяти

6.1 Время работы

Пусть n — число узлов, h — высота дерева. Поиск, вставка и удаление спускаются от корня вниз и на каждом уровне выполняют $O(1)$ действий, поэтому их время пропорционально высоте [1]. Удаление узла с двумя детьми включает до трёх спусков — к удаляемому узлу, к преемнику и повторный спуск при удалении преемника, — так что его время также равно $O(h)$.

Таблица 6.1 – Временная сложность операций улучшенной реализации

Операция	Лучший случай	Средний случай	Худший случай
`insert`, `contains`, `remove`	$O(1)$	$O(\lg n)$	$O(n)$
`preorder`, `inorder`, `postorder`	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
`levelOrder`	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
`height`	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
`print2D` (число вызовов)	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
деструктор	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$

Лучший случай для вставки, поиска и удаления: искомый ключ находится в корне либо дерево пусто, тогда достаточно одного шага.

Средний случай. Для дерева, построенного вставкой ключей в случайном порядке, математическое ожидание высоты равно $O(\lg n)$ [1], поэтому основные операции выполняются в среднем за $O(\lg n)$, а построение всего дерева — за $O(n \lg n)$.

Худший случай. Если ключи вставляются в отсортированном порядке, каждый новый ключ становится потомком предыдущего, дерево превращается в цепочку (пример 7 из раздела 5), $h = n - 1$, и операции выполняются за $O(n)$. Построение всего дерева в этом случае занимает $\sum_{i=1}^{n} i = \Theta(n^2)$.

Все обходы, вычисление высоты и очистка посещают каждый узел ровно один раз, поэтому их время равно $\Theta(n)$ независимо от формы дерева (теорема из раздела «Краткая теория»). Метод `print2D` также делает $\Theta(n)$ вызовов, но объём выводимого текста растёт с глубиной узлов, так как перед ключом печатается $4d$ пробелов; для цепочки из n узлов он составляет $\Theta(n^2)$ символов.

Обход по уровням в реализации Simple Snippets работает за $O(n \cdot h)$, то есть до $O(n^2)$ в худшем случае, а в улучшенной — за $\Theta(n)$.

6.2 Расход памяти

— **Узлы.** Каждый узел хранит ключ `int` (4 байта) и два указателя. На 64-разрядной платформе `sizeof(Node)` равен 24 байтам (4 байта ключа, 4 байта выравнивания и $2 \cdot 8$ байт указателей); к этому добавляются накладные расходы аллокатора. Всего потребуется $\Theta(n)$ памяти, объект класса `BST` дополнительно содержит один указатель.

— **Рекурсия.** Вставка, удаление, обходы, вычисление высоты, вывод и очистка используют стек вызовов глубиной $h + 1$, то есть $O(h)$ дополнительной памяти: $O(\lg n)$ в среднем и $O(n)$ в худшем случае. Размер стека ограничен (порядка мегабайт, в зависимости от системы), поэтому на цепочке из очень большого числа узлов рекурсивные операции завершатся аварийно. Поиск итеративный и требует $O(1)$ памяти.

— **Обход по уровням.** Очередь одновременно содержит узлы не более чем двух соседних уровней, поэтому её размер $O(w)$, где w — максимальная ширина дерева, и в худшем случае $O(n)$.

— **Версия Simple Snippets.** Память под узлы расходуется так же, но каждая итерация меню, кроме успешной вставки, теряет 24 байта, а при выходе не освобождается всё дерево, то есть утечка не ограничена сверху.

6.3 Сравнение реализаций

Таблица 6.2 – Сравнение трёх реализаций

Возможность	Наивная	Simple Snippets	Улучшенная
Вставка и поиск	+	+	+
Удаление	—	+ (с ошибкой)	+
Прямой и обратный обходы	—	+	+
Обход по уровням за $O(n)$	—	— ($O(nh)$)	+
Освобождение памяти	+	—	+
Проверка ввода	—	—	+
Защита от копирования	+	—	+
Балансировка	—	—	—

6.4 Вывод

Время основных операций определяется высотой дерева, а высота зависит от порядка вставки ключей: от $\lg n$ при случайном порядке до n при упорядоченном. Улучшенная реализация устраняет ошибки и утечки памяти версии Simple Snippets и снижает сложность обхода по уровням, но гарантировать логарифмическую высоту не может. Эту задачу решают красно-чёрные и АВЛ-деревья.

ЗАКЛЮЧЕНИЕ

В ходе работы изучено двоичное дерево поиска: его определение и свойство, основные операции (поиск, вставка, удаление) и три вида обхода. Показано, что время основных операций пропорционально высоте дерева, а высота зависит от порядка вставки ключей.

Реализованы и сопоставлены три версии на языке C++: наивная (вставка, поиск, вывод), версия Simple Snippets (удаление, три вида обхода, обход по уровням, высота) и улучшенная. В версии Simple Snippets найдены и проверены запуском недостатки: потеря корня при удалении (обращение к освобождённой памяти), утечки памяти, зацикливание при неверном вводе, обход по уровням за $O(n \cdot h)$ и другие. Улучшенная версия их устраняет: указатель передаётся по ссылке, память создаётся и освобождается только классом, ввод проверяется, обход по уровням выполняется с помощью очереди за $O(n)$.

Оценка сложности показала, что поиск, вставка и удаление выполняются за $O(h)$, то есть за $O(\lg n)$ в среднем и $O(n)$ в худшем случае, а обходы и вычисление высоты — за $\Theta(n)$. Память составляет $\Theta(n)$ на узлы (24 байта на узел при 64-разрядной архитектуре) и $O(h)$ на стек вызовов. Пример с упорядоченной вставкой ключей показал вырождение дерева в цепочку, поэтому в следующих работах будут рассмотрены самобалансирующиеся деревья: красно-чёрное и АВЛ-дерево.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 Алгоритмы: построение и анализ [Текст] / Кормен, Т., Лейзерсон, Ч., Ривест, Р. и Штайн, К. — 3-е изд. — М. : Вильямс, 2013.

2 Седжвик, Р. Фундаментальные алгоритмы на C++ [Текст]. — СПб. : ДиаСофт, 2002.

3 Дерево поиска, наивная реализация: конспект ИТМО [Электронный ресурс онлайн]. — [Б. м. : б. и.]. — Режим доступа: https://neerc.ifmo.ru/wiki/index.php?title=%D0%94%D0%B5%D1%80%D0%B5%D0%B2%D0%BE_%D0%BF%D0%BE%D0%B8%D1%81%D0%BA%D0%B0,_%D0%BD%D0%B0%D0%B8%D0%B2%D0%BD%D0%B0%D1%8F_%D1%80%D0%B5%D0%B0%D0%BB%D0%B8%D0%B7%D0%B0%D1%86%D0%B8%D1%8F (дата обращения: 01.05.2026). Загл. с экр. Яз. рус.

4 Что такое структура данных «Бинарное дерево поиска» (BST)? Все операции с BST и полный код [Электронный ресурс онлайн]. — [Б. м. : б. и.]. — Режим доступа: https://simplesnippets.tech/what-is-binary-search-tree-bst-with-full-code-dsa/ (дата обращения: 01.05.2026). Загл. с экр. Яз. англ.